COMPETITION MASTER 02 — المزرعة الذكية الكاملة على ESP32

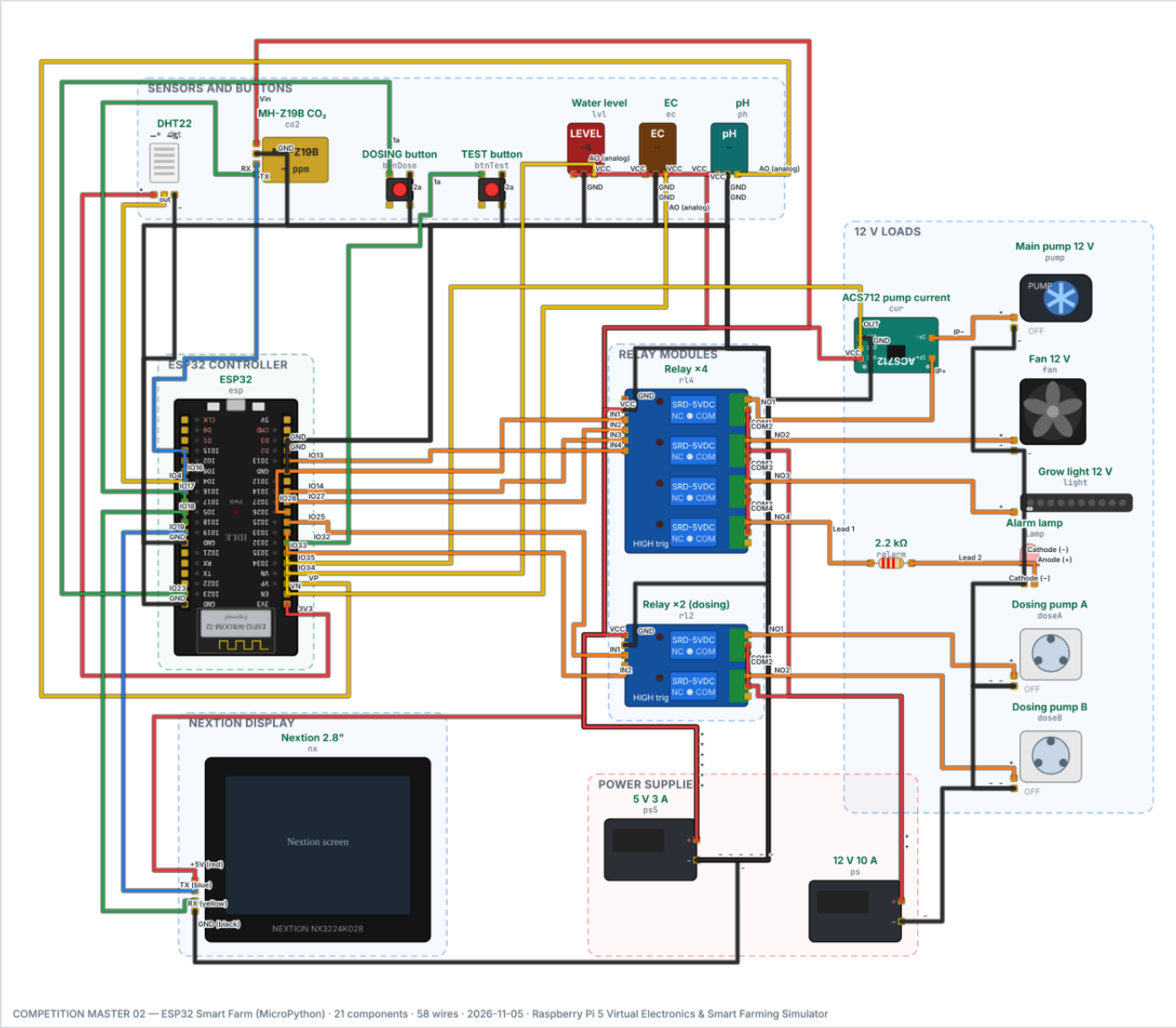
ملف الدراسة الكامل

نفس المزرعة ونفس هيكل برنامج Raspberry Pi، مكتوباً باستخدام machine.Pin وADC وUART، مع عرض الحالة على شاشة المنفذ التسلسلي وعلى شاشة Nextion.

المتحكم	ESP32-DevKitC V4 (ESP32-WROOM-32)
اللغة	MicroPython 1.29 (machine, dht, time)
ملف البرنامج	esp/main.py
عرض الحالة	شاشة المنفذ التسلسلي (أسطر STATUS) + شاشة Nextion (تبويب Nextion). لا يوجد خادم ويب.
القطع / الأسلاك	21 / 58
تاريخ التصدير	2026-11-05

DHT22 (GPIO4)، وMH-Z19B (UART2)، وpH وEC والمستوى وACS712 على ADC1 (GPIO36/39/34/35)، وNextion (UART1)، ومرحّلات على GPIO26/27/14/13/25/33، وأزرار على GPIO23/32، ومصدرا 5 و12 فولت.

1. مخطط الدائرة



21 قطعة · 58 سلك

2. جدول التوصيلات — شرح كل سلك

#	من	إلى	النوع	لماذا	لو انفصل
1	5 V 3 A · –	ESP32 · GND	أرضي	أرضي مشترك: يجب أن تشترك أجزاء 5 فولت والمتحكم في الأرضي حتى يكون لإشاراتها المرجع نفسه	تفقد الأجزاء على هذا السلك 5 فولت؛ وبدون سلك الأرضي المشترك تفقد كل الإشارات مرجعها وتصبح القراءات خاطئة
2	ESP32 · 3V3	DHT22 · +	طاقة	تغذية DHT22	يفقد DHT22 التغذية — لا رد من DHT22: الحرارة None، تضيع القراءة: إنذار SENSOR ERROR ويضيء مصباح الإنذار، والمروحة تعمل للأمان
3	DHT22 · out	ESP32 · IO4	بيانات رقمية	DHT22: خط بيانات رقمي واحد إلى ESP32	لا رد من DHT22: الحرارة None، تضيع القراءة: إنذار SENSOR ERROR ويضيء مصباح الإنذار، والمروحة تعمل للأمان
4	DHT22 · –	ESP32 · GND	أرضي	الأرضي (مسار العودة) لـDHT22	يفقد DHT22 التغذية — لا رد من DHT22: الحرارة None، تضيع القراءة: إنذار SENSOR ERROR ويضيء مصباح الإنذار، والمروحة تعمل للأمان
5	5 V 3 A · +	MH-Z19B CO_2 · Vin	طاقة	تغذية حساس MH-Z19B CO_2	يفقد حساس MH-Z19B CO_2 التغذية — لا رد على الأمر 0x86: co2 = None، تضيع القراءة: إنذار SENSOR ERROR ويضيء مصباح الإنذار
6	MH-Z19B CO_2 · GND	5 V 3 A · –	أرضي	الأرضي (مسار العودة) لـحساس MH-Z19B CO_2	يفقد حساس MH-Z19B CO_2 التغذية — لا رد على الأمر 0x86: co2 = None، تضيع القراءة: إنذار SENSOR ERROR ويضيء مصباح الإنذار
7	ESP32 · IO17	MH-Z19B CO_2 · RX	بيانات رقمية	حساس CO_2 MH-Z19B: UART TX: الحساس ← RX المتحكم، وRX الحساس ← TX المتحكم (متقاطعان)	لا رد على الأمر 0x86: co2 = None، تضيع القراءة: إنذار SENSOR ERROR ويضيء مصباح الإنذار
8	MH-Z19B CO_2 · TX	ESP32 · IO16	بيانات رقمية	حساس CO_2 MH-Z19B: UART TX: الحساس ← RX المتحكم، وRX الحساس ← TX المتحكم (متقاطعان)	لا رد على الأمر 0x86: co2 = None، تضيع القراءة: إنذار SENSOR ERROR ويضيء مصباح الإنذار
9	5 V 3 A · +	pH · VCC	طاقة	تغذية وحدة حساس pH	يفقد وحدة حساس pH التغذية — تُقرأ pH خطأً (قريبة من 0 ← أقل من 1): SENSOR ERROR
10	pH · GND	5 V 3 A · –	أرضي	الأرضي (مسار العودة) لـوحدة حساس pH	يفقد وحدة حساس pH التغذية — تُقرأ pH خطأً (قريبة من 0 ← أقل من 1): SENSOR ERROR
11	pH · VCC	EC · VCC	طاقة	تغذية وحدة حساس pH	يفقد وحدة حساس pH التغذية — تُقرأ pH خطأً (قريبة من 0 ← أقل من 1): SENSOR ERROR
12	EC · GND	pH · GND	أرضي	الأرضي (مسار العودة) لـوحدة حساس pH	يفقد وحدة حساس pH التغذية — تُقرأ pH خطأً (قريبة من 0 ← أقل من 1): SENSOR ERROR
13	EC · VCC	Water level · VCC	طاقة	تغذية وحدة حساس EC	يفقد وحدة حساس EC التغذية — تُقرأ EC خطأً (قريبة من 0): OUT OF RANGE
14	Water level · GND	EC · GND	أرضي	الأرضي (مسار العودة) لـوحدة حساس EC	يفقد وحدة حساس EC التغذية — تُقرأ EC خطأً (قريبة من 0): OUT OF RANGE
15	pH · AO (analog)	ESP32 · VP	إشارة تناظرية	وحدة حساس pH: جهد تناظري: pH = 3.5 × V	تُقرأ pH خطأً (قريبة من 0 ← أقل من 1): SENSOR ERROR
16	EC · AO (analog)	ESP32 · VN	إشارة تناظرية	وحدة حساس EC: جهد تناظري: EC = V ÷ 0.5	تُقرأ EC خطأً (قريبة من 0): OUT OF RANGE
17	Water level · AO (analog)	ESP32 · IO34	إشارة تناظرية	حساس مستوى الماء: جهد تناظري: 3 فولت = 100٪	يُقرأ المستوى خطأً (قريباً من 0٪): LOW WATER ويُرفض الري والجرعات
18	5 V 3 A · +	ACS712 pump current · VCC	طاقة	تغذية حساس التيار ACS712	يفقد حساس التيار ACS712 التغذية — يُقرأ تيار المضخة خطأً: الري التالي يبلّغ عن PUMP FAILURE
19	ACS712 pump current · GND	5 V 3 A · –	أرضي	الأرضي (مسار العودة) لـحساس التيار ACS712	يفقد حساس التيار ACS712 التغذية — يُقرأ تيار المضخة خطأً: الري التالي يبلّغ عن PUMP FAILURE
20	ACS712 pump current · OUT	ESP32 · IO35	إشارة تناظرية	حساس التيار ACS712: جهد تناظري: 2.5 فولت + 0.185 فولت لكل أمبير من تيار المضخة	يُقرأ تيار المضخة خطأً: الري التالي يبلّغ عن PUMP FAILURE
21	5 V 3 A · +	Nextion 2.8" · +5V (red)	طاقة	تغذية شاشة Nextion	يفقد شاشة Nextion التغذية — تتوقف الشاشة عن التحديث؛ وتستمر المزرعة بالعمل (البرنامج يرسل فقط)

#	من	إلى	النوع	لماذا	لو انفصل
22	Nextion 2.8" · GND (black)	5 V 3 A · –	أرضي	الأرضي (مسار العودة) لـشاشة Nextion	يفقد شاشة Nextion التغذية — تتوقف الشاشة عن التحديث؛ وتستمر المزرعة بالعمل (البرنامج يرسل فقط)
23	ESP32 · IO18	Nextion 2.8" · RX (yellow)	بيانات رقمية	شاشة Nextion: UART TX1 في ESP32 ← RX الشاشة (أوامر)؛ وTX الشاشة ← RX1 (أحداث اللمس، غير مستخدمة هنا)	تتوقف الشاشة عن التحديث؛ وتستمر المزرعة بالعمل (البرنامج يرسل فقط)
24	Nextion 2.8" · TX (blue)	ESP32 · IO19	بيانات رقمية	شاشة Nextion: UART TX1 في ESP32 ← RX الشاشة (أوامر)؛ وTX الشاشة ← RX1 (أحداث اللمس، غير مستخدمة هنا)	تتوقف الشاشة عن التحديث؛ وتستمر المزرعة بالعمل (البرنامج يرسل فقط)
25	5 V 3 A · +	Relay ×4 · VCC	طاقة	تغذية وحدة مرحّل 4 قنوات	تفقد ملفات المرحّل التغذية: لا تعمل المضخة ولا المروحة ولا الإضاءة ولا مصباح الإنذار
26	Relay ×4 · GND	5 V 3 A · –	أرضي	الأرضي (مسار العودة) لـوحدة مرحّل 4 قنوات	تفقد ملفات المرحّل التغذية: لا تعمل المضخة ولا المروحة ولا الإضاءة ولا مصباح الإنذار
27	ESP32 · IO26	Relay ×4 · IN1	بيانات رقمية	وحدة مرحّل 4 قنوات: مدخل المرحّل: المستوى HIGH يغلق القناة	لا يتبع المرحّل البرنامج: يطيع البرنامج pump=ON بينما يبقى الحمل متوقفاً
28	ESP32 · IO27	Relay ×4 · IN2	بيانات رقمية	وحدة مرحّل 4 قنوات: مدخل المرحّل: المستوى HIGH يغلق القناة	لا يتبع المرحّل البرنامج: يطيع البرنامج pump=ON بينما يبقى الحمل متوقفاً
29	ESP32 · IO14	Relay ×4 · IN3	بيانات رقمية	وحدة مرحّل 4 قنوات: مدخل المرحّل: المستوى HIGH يغلق القناة	لا يتبع المرحّل البرنامج: يطيع البرنامج pump=ON بينما يبقى الحمل متوقفاً
30	ESP32 · IO13	Relay ×4 · IN4	بيانات رقمية	وحدة مرحّل 4 قنوات: مدخل المرحّل: المستوى HIGH يغلق القناة	لا يتبع المرحّل البرنامج: يطيع البرنامج pump=ON بينما يبقى الحمل متوقفاً
31	5 V 3 A · +	Relay ×2 (dosing) · VCC	طاقة	تغذية وحدة مرحّل قناتين (الجرعات)	يفقد مرحّلا الجرعات التغذية: لا جرعات أبداً
32	Relay ×2 (dosing) · GND	5 V 3 A · –	أرضي	الأرضي (مسار العودة) لـوحدة مرحّل قناتين (الجرعات)	يفقد مرحّلا الجرعات التغذية: لا جرعات أبداً
33	ESP32 · IO25	Relay ×2 (dosing) · IN1	بيانات رقمية	وحدة مرحّل قناتين (الجرعات): مدخل المرحّل: المستوى HIGH يغلق القناة	لا تعمل مضخة الجرعات رغم أن البرنامج يطيع DOSING …
34	ESP32 · IO33	Relay ×2 (dosing) · IN2	بيانات رقمية	وحدة مرحّل قناتين (الجرعات): مدخل المرحّل: المستوى HIGH يغلق القناة	لا تعمل مضخة الجرعات رغم أن البرنامج يطيع DOSING …
35	ACS712 pump current · IP–	Main pump 12 V · +	دائرة الحمل (12 فولت)	دائرة 12 فولت لـالمضخة الرئيسية: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تعمل المضخة: لا تيار في ACS712 ← PUMP FAILURE: no current
36	2.2 kΩ · Lead 2	Alarm lamp · Anode (+)	بيانات رقمية	مصباح الإنذار: مصباح 12 فولت (LED + 2.2 كيلو أوم)	تُطبع الإنذارات لكن المصباح لا يضيء أبداً
37	12 V 10 A · +	Relay ×4 · COM1	دائرة الحمل (12 فولت)	دائرة 12 فولت لـالمضخة الرئيسية: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تعمل المضخة: لا تيار في ACS712 ← PUMP FAILURE: no current
38	Relay ×4 · NO1	ACS712 pump current · IP+	دائرة الحمل (12 فولت)	دائرة 12 فولت لـالمضخة الرئيسية: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تعمل المضخة: لا تيار في ACS712 ← PUMP FAILURE: no current
39	Relay ×4 · COM1	Relay ×4 · COM2	دائرة الحمل (12 فولت)	دائرة 12 فولت لـالمروحة: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تعمل المروحة: يستمر هواء المزرعة بالسخونة، والبرنامج لا يعرف ذلك (لا توجد تغذية راجعة للمروحة)
40	Relay ×4 · NO2	Fan 12 V · +	دائرة الحمل (12 فولت)	دائرة 12 فولت لـالمروحة: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تعمل المروحة: يستمر هواء المزرعة بالسخونة، والبرنامج لا يعرف ذلك (لا توجد تغذية راجعة للمروحة)
41	Relay ×4 · COM2	Relay ×4 · COM3	دائرة الحمل (12 فولت)	دائرة 12 فولت لـإضاءة النمو: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	تبقى الإضاءة مطفأة رغم أن البرنامج يطيع light=ON
42	Relay ×4 · NO3	Grow light 12 V · +	دائرة الحمل (12 فولت)	دائرة 12 فولت لـإضاءة النمو: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	تبقى الإضاءة مطفأة رغم أن البرنامج يطيع light=ON
43	Relay ×4 · COM3	Relay ×4 · COM4	دائرة الحمل (12 فولت)	دائرة 12 فولت لـمصباح الإنذار: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	تُطبع الإنذارات لكن المصباح لا يضيء أبداً
44	Relay ×4 · NO4	2.2 kΩ · Lead 1	دائرة الحمل (12 فولت)	دائرة 12 فولت لـمصباح الإنذار: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	تُطبع الإنذارات لكن المصباح لا يضيء أبداً
45	12 V 10 A · +	Relay ×2 (dosing) · COM1	دائرة الحمل (12 فولت)	دائرة 12 فولت لـمضخة الجرعات A: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تُضخ الجرعة A رغم طباعة DOSING A (تبقى مجاميع المزرعة 0 مل)

#	من	إلى	النوع	لماذا	لو انفصل
46	Relay ×2 (dosing) · NO1	Dosing pump A · +	دائرة الحمل (12 فولت)	دائرة 12 فولت لـمضخة الجرعات A: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تُضخ الجرعة A رغم طباعة DOSING A (تبقى مجاميع المزرعة 0 مل)
47	Relay ×2 (dosing) · COM1	Relay ×2 (dosing) · COM2	دائرة الحمل (12 فولت)	دائرة 12 فولت لـمضخة الجرعات B: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تُضخ الجرعة B رغم طباعة DOSING B
48	Relay ×2 (dosing) · NO2	Dosing pump B · +	دائرة الحمل (12 فولت)	دائرة 12 فولت لـمضخة الجرعات B: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تُضخ الجرعة B رغم طباعة DOSING B
49	Main pump 12 V · –	Fan 12 V · –	دائرة الحمل (12 فولت)	دائرة 12 فولت لـالمضخة الرئيسية: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تعمل المضخة: لا تيار في ACS712 ← PUMP FAILURE: no current
50	Fan 12 V · –	Grow light 12 V · –	دائرة الحمل (12 فولت)	دائرة 12 فولت لـالمروحة: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تعمل المروحة: يستمر هواء المزرعة بالسخونة، والبرنامج لا يعرف ذلك (لا توجد تغذية راجعة للمروحة)
51	Grow light 12 V · –	Alarm lamp · Cathode (–)	دائرة الحمل (12 فولت)	دائرة 12 فولت لـإضاءة النمو: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	تبقى الإضاءة مطفأة رغم أن البرنامج يطبع light=ON
52	Alarm lamp · Cathode (–)	Dosing pump A · –	دائرة الحمل (12 فولت)	دائرة 12 فولت لـمضخة الجرعات A: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تُضخ الجرعة A رغم طباعة DOSING A (تبقى مجاميع المزرعة 0 مل)
53	Dosing pump A · –	Dosing pump B · –	دائرة الحمل (12 فولت)	دائرة 12 فولت لـمضخة الجرعات A: موجب المصدر ← تلامس المرحّل ← الحمل ← سالب المصدر	لا تُضخ الجرعة A رغم طباعة DOSING A (تبقى مجاميع المزرعة 0 مل)
54	Dosing pump B · –	12 V 10 A · –	أرضي	الأرضي (مسار العودة) لـمضخة الجرعات B	يفقد مضخة الجرعات B التغذية — لا تُضخ الجرعة B رغم طباعة DOSING B
55	ESP32 · IO23	DOSING button · 1a	بيانات رقمية	زر DOSING: مدخل بمقاومة سحب داخلية: مضغوط = موصول بالأرضي = 0	لا يعمل الزر (مقاومة السحب تبقي المدخل "غير مضغوط")
56	DOSING button · 2a	ESP32 · GND	أرضي	الأرضي (مسار العودة) لـزر DOSING	يفقد زر DOSING التغذية — لا يعمل الزر (مقاومة السحب تبقي المدخل "غير مضغوط")
57	ESP32 · IO32	TEST button · 1a	بيانات رقمية	زر TEST: مدخل بمقاومة سحب داخلية: مضغوط = موصول بالأرضي = 0	لا يعمل الزر (مقاومة السحب تبقي المدخل "غير مضغوط")
58	TEST button · 2a	ESP32 · GND	أرضي	الأرضي (مسار العودة) لـزر TEST	يفقد زر TEST التغذية — لا يعمل الزر (مقاومة السحب تبقي المدخل "غير مضغوط")

3. قائمة القطع

#	المكون	الموديل / المرجع	الكمية	الخصائص المهمة	الجهد	مستوى المحاكاة	المعرّفات	ملاحظات التركيب
1	ESP32-DevKitC V4 (ESP32-WROOM-32)	ESP32-DevKitC V4 · Espressif	1	Power source: Micro-USB from the PC (5 V)	5 V USB or 5V pin (on-board 3.3 V regulator); GPIO 3.3 V, not 5 V tolerant	نموذج وظيفي	esp	
2	DC power supply	Bench / adapter DC supply	1	Voltage: 5 V; Current limit: 3 A	—	محاكاة كاملة	ps5	
3	DHT22 temperature/humidity	DHT22 / AM2302 module · Aosong	1	Module with pull-up resistor: yes	3.3–6 V	محاكاة كاملة	dht	
4	MH-Z19B CO_2 sensor	MH-Z19B · Winsen	1	—	4.5–5.5 V supply, 3.3 V UART	محاكاة كاملة	co2	
5	pH sensor (analog)	Analog pH probe + board (SEN0161-style)	1	—	3.3–5 V	نموذج وظيفي	ph	
6	EC conductivity sensor (analog)	Analog EC probe + board	1	—	3.3–5 V	نموذج وظيفي	ec	
7	Water level sensor (analog)	Analog water-level sensor	1	—	3.3–5 V	نموذج وظيفي	lvl	
8	ACS712 current sensor (5 A)	ACS712ELCTR-05B · Allegro	1	Version: 5 A (185 mV/A)	5 V supply	محاكاة كاملة	cur	
9	Nextion Enhanced 2.8" (NX3224K028)	NX3224K028 · ITEAD	1	Baud rate (bauds): 9600	4.75–7 V supply, 3.3 V TX (5 V-tolerant RX)	محاكاة كاملة	nx	
10	Relay module	SRD-05VDC-SL-C opto relay module	1	Channels: 4; Trigger: Active-HIGH (IN=1 → ON); Coil voltage: 5 V	5 V or 12 V coil, 3.3/5 V logic	محاكاة كاملة	rl4	
11	Relay module	SRD-05VDC-SL-C opto relay module	1	Channels: 2; Trigger: Active-HIGH (IN=1 → ON); Coil voltage: 5 V	5 V or 12 V coil, 3.3/5 V logic	محاكاة كاملة	rl2	
12	DC power supply	Bench / adapter DC supply	1	Voltage: 12 V; Current limit: 10 A	—	محاكاة كاملة	ps	
13	Water pump (12 V DC)	12 V DC pump	1	Rated power: 6 W	12 V	نموذج وظيفي	pump	
14	Cooling fan (DC)	12 V DC fan	1	Rated power: 2.4 W	12 V	نموذج وظيفي	fan	
15	LED grow light	12 V LED grow bar	1	Rated power: 36 W	12 V	نموذج وظيفي	light	
16	Resistor	¼ W carbon-film resistor	1	Resistance: 2.2 kΩ	—	محاكاة كاملة	ralarm	
17	LED	5 mm LED	1	Colour: red	Vf ≈ 1.9–2.9 V, 20 mA max	محاكاة كاملة	lamp	
18	Peristaltic dosing pump	12 V peristaltic pump	2	Rated power: 3.6 W	12 V	نموذج وظيفي	doseA, doseB	
19	Push button	6×6 mm tactile switch	2	—	—	محاكاة كاملة	btnDose, btnTest	
20	أسلاك توصيل	—	58	18× أسود, 15× أحمر, 5× أصفر, 4× أخضر, 2× أزرق, 14× برتقالي	—	—		استخدم أسلاك ذكر–أنثى لموصل Pi، واجعل أسلاك التغذية قصيرة.

4. الإعدادات (الثوابت في أعلى البرنامج)

الاسم	القيمة	المعنى
MAX_TEMP	30	فوق 30 درجة: تشغيل المروحة
FAN_OFF_TEMP	27	تحت 27 درجة: إيقاف المروحة
MIN_WATER	20	أقل مستوى ماء مسموح (%) لتشغيل المضخة
PUMP_SECONDS	5	مدة الري بالثواني
IRRIGATION_HOURS	[7, 12, 17]	ساعات الري في اليوم
LIGHT_ON_HOUR	6	ساعة تشغيل الإضاءة
LIGHT_OFF_HOUR	20	ساعة إطفاء الإضاءة
DOSE_A_ML	10	جرعة السماد A بالمليلتر
DOSE_B_ML	5	جرعة السماد B بالمليلتر
FLOW_ML_PER_SEC	2	تدفق مضخة الجرعات (يحتاج معايرة على الجهاز الحقيقي)
PH_MIN	5.5	أقل pH مقبول
PH_MAX	6.5	أعلى pH مقبول
EC_MIN	1.0	أقل EC مقبول (mS/cm)
EC_MAX	2.5	أعلى EC مقبول
MIN_PUMP_AMPS	0.2	أقل تيار يثبت أن المضخة تعمل فعلاً (أمبير)
TEST_SECONDS	2	مدة تشغيل كل جهاز في وضع الاختبار

قيم تدريبية يمكنك تغييرها، وليست إعدادات معتمدة لكل المحاصيل. دقة الجرعات تعتمد على معايرة المضخات الحقيقية.

5. البرنامج في 9 أقسام

1 · التهيئة والتوصيلات

نفس إعدادات machine.Pin في Pi. تأخذ أرقام GPIO في ESP32؛ الحساسات التناظرية على أرجل ADC1 (36 و39 و34 و35) التي تعمل مع Wi-Fi.

- غيّر الحد هنا وسيتبعه البرنامج كله.
- GPIO34–39 للإدخال فقط؛ لا تستخدمها كمخارج.

esp/main.py

```
---------- Raspberry Pi مشروع قيم نفس :الإعدادات (1 ---------- #  7
MAX_TEMP = 30  8                # فوق 30 درجة: تشغيل المروحة
FAN_OFF_TEMP = 27  9            # تحت 27 درجة: إيقاف المروحة
MIN_WATER = 20  10              # أقل مستوى ماء مسموح (%) لتشغيل المضخة
PUMP_SECONDS = 5  11            # مدة الري بالثواني
IRRIGATION_HOURS = [7, 12, 17]  12  # ساعات الري في اليوم
LIGHT_ON_HOUR = 6  13           # ساعة تشغيل الإضاءة
LIGHT_OFF_HOUR = 20  14         # ساعة إطفاء الإضاءة
DOSE_A_ML = 10  15              # جرعة السماد A بالمليلتر
DOSE_B_ML = 5  16               # جرعة السماد B بالمليلتر
FLOW_ML_PER_SEC = 2  17         # تدفق مضخة الجرعات (يحتاج معايرة على الجهاز الحقيقي)
PH_MIN = 5.5  18                # أقل pH مقبول
PH_MAX = 6.5  19                # أعلى pH مقبول
EC_MIN = 1.0  20                # أقل EC مقبول (mS/cm)
EC_MAX = 2.5  21                # أعلى EC مقبول
CO2_MAX = 1500  22              # أعلى CO2 مقبول (ppm)
MIN_PUMP_AMPS = 0.2  23         # أقل تيار يثبت أن المضخة تعمل فعلاً (أمبير)
TEST_SECONDS = 2  24            # مدة تشغيل كل جهاز في وضع الاختبار
 25
 26  # >>> PINS
---------- التوصيلات: كل جهاز على رجله (2 ---------- #  27
dht_sensor = dht.DHT22(Pin(4))                      # DHT22  28  :البيانات على GPIO4
UART2 (TX=17، RX=16) على co2_port = UART(2, baudrate=9600, tx=17, rx=16, timeout=300)  # MH-Z19B  29
UART1 (TX=18، RX=19) على Nextion شاشة #       screen = UART(1, baudrate=9600, tx=18, rx=19)  30
ph_adc = ADC(Pin(36), atten=ADC.ATTN_11DB)          # pH  31  GPIO36 (ADC1 على Wi-Fi، حتى 3.3 فولت يعمل مع
↪ تقريباً)
ec_adc = ADC(Pin(39), atten=ADC.ATTN_11DB)          # EC  32  GPIO39 (ADC1) على
```

```
GPIO34 (ADC1) على مستوى الماء #       level_adc = ADC(Pin(34), atten=ADC.ATTN_11DB)  33
GPIO35 (ADC1) على ACS712 تيار المضخة #     current_adc = ADC(Pin(35), atten=ADC.ATTN_11DB)  34
(يبدأ مطفأً) GPIO26 على مرحّل مضخة الري #                    pump = Pin(26, Pin.OUT, value=0)  35
GPIO27 على مرحّل المروحة #                    fan = Pin(27, Pin.OUT, value=0)  36
GPIO14 على مرحّل الإضاءة #                   light = Pin(14, Pin.OUT, value=0)  37
GPIO13 على مرحّل مصباح الإنذار #              alarm_lamp = Pin(13, Pin.OUT, value=0)  38
GPIO25 على A مرحّل مضخة الجرعات #                 dose_a = Pin(25, Pin.OUT, value=0)  39
GPIO33 على B مرحّل مضخة الجرعات #                 dose_b = Pin(33, Pin.OUT, value=0)  40
(إلى GND، مقاومة سحب داخلية) GPIO23 على زر الجرعات #        dosing_button = Pin(23, Pin.IN, Pin.PULL_UP)  41
(إلى GND) GPIO32 على زر الاختبار #          test_button = Pin(32, Pin.IN, Pin.PULL_UP)  42
 43  # <<< PINS
```

2 · قراءة الحساسات

دالة واحدة تقرأ كل شيء وتحفظه في متغيرات. القراءة المفقودة تصبح None/null حتى تراها الإنذارات. الحساسات التناظرية تعطي جهداً؛ ومعادلة لكل حساس تحوّل الجهد إلى pH وEC ونسبة مئوية وأمبير.

- pH = 3.5 − V × 0.5 ÷ V · EC = V المستوى = 100 × 3 ÷ V · التيار = (V − 2.5) ÷ 0.185 (وحدات المحاكي؛ عايِر الحساسات الحقيقية).
- يحتاج DHT22 ثانيتين بين القراءات: البرنامج يقرؤه مرة كل ثانيتين على الأكثر.

esp/main.py

```
UART عبر MH-Z19B من CO2 قراءة #                                  :()def read_co2  65
(9 بايت) أمر طلب القراءة #  co2_port.write(bytes([0xFF, 0x01, 0x86, 0, 0, 0, 0, 0, 0x79]))      66
(أو None بعد المهلة) نستظر رداً من 9 بايت #                      reply = co2_port.read(9)      67
لا رد = الحساس مفصول #           :if reply is None or len(reply) != 9      68
تعني: لا توجد قراءة return None                                  # None          69
ppm التركيز = 3 و البايت 2 #                return reply[2] * 256 + reply[3]      70
 73  # >>> SENSORS
قراءة كل الحساسات #                                  :()def read_sensors  74
سنغيّر هذه #  global temperature, humidity, co2, ph, ec, water_level, pump_amps, last_dht_time      75
↪ المتغيرات العامة
قراءة جديدة كل ثانيتين فقط :if time.time() - last_dht_time >= 2:            # DHT22      76
نحفظ وقت هذه القراءة #                ()last_dht_time = time.time          77
نحاول القراءة #                                        :try          78
نطلب القياس أولاً MicroPython في #                    ()dht_sensor.measure              79
C° الحرارة #  ()temperature = dht_sensor.temperature              80
% الرطوبة #        ()humidity = dht_sensor.humidity              81
(الحساس مفصول) فشل القراءة #                                :except OSError          82
لا توجد قراءة حرارة #                     temperature = None              83
ولا رطوبة #                          humidity = None              84
(None أو) co2 = read_co2()                                    # CO2 ppm      85
الجهد × Raspberry Pi: pH = 3.5 نفس معادلة #              ph = round(volts(ph_adc) * 3.5, 2)      86
0.5 ÷ الجهد = ec = round(volts(ec_adc) / 0.5, 2)              # EC      87
%100 = فولت water_level = round(volts(level_adc) / 3.0 * 100)  # 3      88
صفر أمبير = فولت pump_amps = round((volts(current_adc) - 2.5) / 0.185, 2)  # ACS712: 2.5      89
 90  # <<< SENSORS
```

3 · تشغيل المخرجات

نفس فكرة Pi: Pin.on()/off() تشغّل مداخل المرحّلات (منطق 3.3 فولت، وحدة تعمل بـHIGH)؛ والمرحّلات توصل أحمال 12 فولت.

- الأحمال: المضخة والمروحة والإضاءة ومضخات الجرعات على مصدر 12 فولت، وليس على أرجل المتحكم أبداً.
- مصباح الإنذار مصباح 12 فولت على قناة مرحّل خاصة به.

esp/main.py

```
(يبدأ مطفأً) GPIO26 على مرحّل مضخة الري #                    pump = Pin(26, Pin.OUT, value=0)  35
GPIO27 على مرحّل المروحة #                    fan = Pin(27, Pin.OUT, value=0)  36
GPIO14 على مرحّل الإضاءة #                   light = Pin(14, Pin.OUT, value=0)  37
GPIO13 على مرحّل مصباح الإنذار #              alarm_lamp = Pin(13, Pin.OUT, value=0)  38
GPIO25 على A مرحّل مضخة الجرعات #                 dose_a = Pin(25, Pin.OUT, value=0)  39
GPIO33 على B مرحّل مضخة الجرعات #                 dose_b = Pin(33, Pin.OUT, value=0)  40
لعرض حالة جهاز "OFF" أو "def on_off(device):                                  # "ON  206
0 أو 1 (يعمل) تعطي ()if device.value() == 1:                          # Pin.value      207
يعمل #                                  "return "ON          208
متوقف #                                      "return "OFF      209
```

4 · الشروط والحماية

الدالة water_ok() هي المكان الوحيد الذي يقرر "هل يوجد ماء كافٍ؟". الري يفحصها قبل كل ثانية وأثناءها، ويفحص تيار المضخة: المضخة المفصولة أو العالقة لا تسحب تياراً، فيوقفها البرنامج ويتذكر العطل.

- المستوى فوق 100٪ يُعدّ قراءة غير صالحة وليس "ممتلئاً".
- تُطفأ المضخة دائماً في النهاية مهما كان سبب توقف الري.

esp/main.py

```
def water_ok():  # 93
    return water_level >= MIN_WATER and water_level <= 100  # 94 قراءة غير صالحة = أكثر من 100%
# >>> IRRIGATION  101
def irrigate():  # 102
    global pump_failed  # 103 قد نغيّر هذا المتغير العام
    print("IRRIGATION start")  # 104 رسالة بداية الري
    for second in range(PUMP_SECONDS):  # 105 ثانية بعد ثانية
        read_sensors()  # 106 قراءة جديدة قبل كل ثانية
        if not water_ok():  # 107 الماء نقص أثناء الري
            print("IRRIGATION stopped: LOW WATER")  # 108 نخبر بالسبب
            break  # 109 نخرج من الحلقة فوراً
        pump.on()  # 110 تشغيل المضخة
        time.sleep(1)  # 111 انتظار ثانية
        read_sensors()  # 112 نقرأ تيار المضخة الآن
        if pump_amps < MIN_PUMP_AMPS:  # 113 المضخة لا تسحب تياراً = لا تعمل
            pump_failed = True  # 114 نتذكر العطل (يظهر الإنذار)
            print("PUMP FAILURE: no current")  # 115 رسالة العطل
            break  # 116 نتوقف عن الري
    pump.off()  # 117 إيقاف المضخة دائماً في النهاية
    print("IRRIGATION end")  # 118 رسالة نهاية الري
# <<< IRRIGATION  119
```

5 · وضع AUTO

كل ثانية: المروحة تتبع الحرارة بحدّين (تعمل فوق 30 °م وتتوقف تحت 27 °م — بينهما تبقى على حالها فلا تتقطع)، والإضاءة تتبع الساعة (06:00–20:00)، والري يبدأ مرة واحدة الساعة 07:00 و12:00 و17:00 إذا وُجد ماء.

- لا توجد قراءة حرارة ← تعمل المروحة (الجانب الآمن للنباتات).
- المتغير last_irrigation_hour يمنع تكرار الري خلال الساعة نفسها.

esp/main.py

```
def auto_control():  # 156 الوضع التلقائي
    global last_irrigation_hour  # 157 سنغيّر هذا المتغير العام
    hour = time.localtime()[3]  # 158 الساعة الحالية: في MicroPython هي العنصر رقم 3
# >>> FAN  159
    if temperature is None:  # 160 لا قراءة حرارة: نشغّل المروحة للأمان
        fan.on()  # 161 تشغيل المروحة
    elif temperature > MAX_TEMP:  # 162 حار جداً
        fan.on()  # 163 تشغيل المروحة
    elif temperature < FAN_OFF_TEMP:  # 164 برد بما يكفي
        fan.off()  # 165 إيقاف المروحة (بين 27 و 30 تبقى كما هي)
# <<< FAN  166
# >>> LIGHT  167
    if hour >= LIGHT_ON_HOUR and hour < LIGHT_OFF_HOUR:  # 168 داخل ساعات الإضاءة
        light.on()  # 169 تشغيل الإضاءة
    else:  # 170 خارج ساعات الإضاءة
        light.off()  # 171 إطفاء الإضاءة
# <<< LIGHT  172
    if hour in IRRIGATION_HOURS and hour != last_irrigation_hour:  # 173 حان وقت ري جديد
        last_irrigation_hour = hour  # 174 نتذكر الساعة حتى لا نكرر الري
        if water_ok():  # 175 يوجد ماء؟
            irrigate()  # 176 نروي
        else:  # 177 لا يوجد ماء
            print("IRRIGATION skipped: LOW WATER")  # 178 نتخطى الري
```

6 · وضع DOSING

افحص الماء، أطفئ B، شغّل A لمدة (الجرعة ÷ التدفق) ثانية، أطفئ A، شغّل B، أطفئ B، اطبع DOSING DONE. لا تعمل A وB معاً أبداً. دقة الكميات تعتمد على معايرة تدفق المضخات الحقيقية (هنا 2 مل/ث) وليست مضمونة بالحساب.

- يبدأ بزر DOSING (أو /mode/DOSING من صفحة ويب Pi).
- المزرعة الذكية تعدّ المليلترات التي ضُخّت فعلاً، فيمكنك التحقق من الجرعات.

esp/main.py

```
# >>> DOSING  122
def run_dosing():  # 123 جرعة A ثم جرعة B (لا تعملان معاً أبداً)
```

```
  3 لا تسميد بدون ماء #                             :()if not water_ok      124
  4 نخبر بالسبب #         print("DOSING cancelled: LOW WATER")          125
  5 نخرج من الدالة #                                    return          126
  6 A متوقفة قبل تشغيل B تأكد أن #                                    ()dose_b.off      127
  7 A تشغيل #                                    ()dose_a.on      128
  8 A رسالة الجرعة #              print("DOSING A", DOSE_A_ML, "ml")      129
  9 (10 ÷ 2 = 5 ثوانٍ) الزمن = الكمية ÷ التدفق #         time.sleep(DOSE_A_ML / FLOW_ML_PER_SEC)      130
 10 A إيقاف #                                    ()dose_a.off      131
 11 B ثم تشغيل #                                    ()dose_b.on      132
 12 B رسالة الجرعة #              print("DOSING B", DOSE_B_ML, "ml")      133
 13 ثانية time.sleep(DOSE_B_ML / FLOW_ML_PER_SEC)         # 5 ÷ 2 = 2.5      134
 14 B إيقاف #                                    ()dose_b.off      135
 15 انتهت الجرعات #                             print("DOSING DONE")      136
 16 137  # <<< DOSING
```

7 · وضع TEST

يعمل كل مخرج ثانيتين بالترتيب: المروحة، الإضاءة، الجرعة A، الجرعة B، ثم المضخة — والمضخة فقط إذا وُجد ماء. الحماية تبقى فعّالة في وضع الاختبار.

- استخدمه بعد توصيل مزرعة حقيقية: يجب أن ينقر كل مرحّل مرة ويعمل كل حمل.
- الجرعة A والجرعة B تعملان الواحدة بعد الأخرى، لا معاً.

esp/main.py

```
  1 تجربة كل جهاز لثانيتين #                                        :()def run_test  140
  2 قائمة #  :for name, device in [("FAN", fan), ("LIGHT", light), ("DOSE A", dose_a), ("DOSE B", dose_b)]      141
↪ الأجهزة
  3 اسم الجهاز الذي نختبره #                          print("TEST", name)          142
  4 تشغيله #                                 ()device.on          143
  5 ننتظر ثانيتين #                    time.sleep(TEST_SECONDS)          144
  6 إيقافه #                                 ()device.off          145
  7 المضخة فقط إذا يوجد ماء (الحماية لا تُلغى) #                                 :()if water_ok      146
  8 نختبر المضخة #                           print("TEST PUMP")          147
  9 تشغيلها #                                    ()pump.on          148
 10 ننتظر ثانيتين #                    time.sleep(TEST_SECONDS)          149
 11 إيقافها #                                   ()pump.off          150
 12 لا يوجد ماء كافٍ #                                          :else      151
 13 لا نشغّل المضخة فارغة #       print("TEST PUMP skipped: LOW WATER")          152
 14 انتهى الاختبار #                             print("TEST DONE")      153
```

8 · الإنذارات

سلسلة if/elif واحدة تختار أهم إنذار: SENSOR ERROR ثم LOW WATER ثم PUMP FAILURE ثم HIGH TEMPERATURE ثم OUT OF RANGE، أو NONE. مصباح الإنذار يعمل مع أي إنذار، ويُطبع الإنذار فقط عند تغيّره.

- ترتيب أسطر elif هو الأولوية: الحساس المفقود أهم من ارتفاع الحرارة.
- إنذار PUMP FAILURE يبقى حتى إعادة تشغيل البرنامج (يجب أن يفحص شخص المضخة).

esp/main.py

```
  1 181  # >>> ALARMS
  2 اختيار أهم إنذار وتشغيل المصباح #                                    :()def check_alarms  182
  3 سنغيّر هذا المتغير العام #                                  global alarm      183
  4 نحفظ الإنذار السابق #                                old = alarm      184
  5 قراءة غير صالحة #  :if temperature is None or co2 is None or ph < 1 or ph > 14 or water_level > 100      185
  6 خطأ حساس #                      "alarm = "SENSOR ERROR          186
  7 الماء قليل #                             :()elif not water_ok      187
  8 نقص الماء #                         "alarm = "LOW WATER          188
  9 المضخة لم تسحب تياراً #                                 :elif pump_failed      189
 10 عطل المضخة #                     "alarm = "PUMP FAILURE          190
 11 حرارة عالية #                   :elif temperature > MAX_TEMP      191
 12 ارتفاع الحرارة #                 "alarm = "HIGH TEMPERATURE          192
 13 خارج الحدود #  :elif ph < PH_MIN or ph > PH_MAX or ec < EC_MIN or ec > EC_MAX or co2 > CO2_MAX      193
 14 قيمة خارج المجال #                      "alarm = "OUT OF RANGE          194
 15 كل شيء طبيعي #                                    :else      195
 16 لا يوجد إنذار #                               "alarm = "NONE          196
 17 لا إنذار #                                 :"if alarm == "NONE      197
 18 إطفاء مصباح الإنذار #                                 ()alarm_lamp.off          198
 19 يوجد إنذار #                                       :else      199
 20 تشغيل مصباح الإنذار #                                ()alarm_lamp.on          200
 21 نطبع الإنذار عند تغيّره فقط #                                  :if alarm != old      201
 22 رسالة الإنذار #                        print("ALARM", alarm)          202
 23 203  # <<< ALARMS
```

9 · العرض والمراقبة

كل 5 ثوانٍ يُطبع سطر STATUS في شاشة المنفذ التسلسلي، وتُرسل القيم نفسها إلى شاشة Nextion (tTemp.txt="..." ثم ثلاثة 0xFF). لا يوجد خادم ويب في برنامج ESP32 هذا؛ يمكن إضافته لاحقاً على الجهاز الحقيقي عبر Wi-Fi.

- السطر STATUS mode=... temp=... ... alarm=... day=... هو نفسه في المتحكمات الثلاثة.
- لوحة الحالة في هذه الصفحة تقرأ هذا السطر من البرنامج أثناء تشغيله.

esp/main.py

```
def show(name, text):  # كتابة نص في عنصر على شاشة Nextion
    screen.write(name + '.txt="' + str(text) + '"')  # مثال: "tTemp.txt="24.5
    screen.write(bytes([0xFF, 0xFF, 0xFF]))  # كل أمر Nextion ينتهي بثلاثة 0xFF


def show_status():  # الحالة على Serial وعلى الشاشة
    print("STATUS mode=" + mode, "temp=" + str(temperature), "hum=" + str(humidity), "co2=" + str(co2),  # الوضع والهواء
          ",ph=" + str(ph), "ec=" + str(ec), "level=" + str(water_level), "pump=" + on_off(pump)"  # الماء والمضخة
          "fan=" + on_off(fan), "light=" + on_off(light), "alarm=" + alarm, "day=" + str(growth_day()))  # الباقي
    show("tTemp", temperature)  # الحرارة على الشاشة
    show("tHum", humidity)  # الرطوبة
    show("tCO2", co2)  # CO2
    show("tPH", ph)  # pH
    show("tEC", ec)  # EC
    show("tLevel", water_level)  # مستوى الماء
    show("tMode", mode)  # الوضع
    show("tAlarm", alarm)  # الإنذار
    show("tDay", growth_day())  # يوم النمو


# ---------- الحلقة الرئيسية (4 ----------
print("Smart farm started (ESP32)")  # رسالة البداية على Serial Monitor
loop = 0  # عدّاد الدورات
while True:  # تكرار إلى الأبد
    read_sensors()  # قراءة الحساسات
    if dosing_button.value() == 0:  # الزر المضغوط يقرأ 0 (موصول إلى GND)  # 2
        mode = "DOSING"  # نختار وضع الجرعات
    if test_button.value() == 0:  # زر الاختبار مضغوط؟
        mode = "TEST"  # نختار وضع الاختبار
    if mode == "DOSING":  # تنفيذ الوضع
        run_dosing()  # الجرعات A ثم B
        mode = "AUTO"  # ثم العودة إلى التلقائي
    elif mode == "TEST":  # وضع الاختبار
        run_test()  # تجربة الأجهزة
        mode = "AUTO"  # ثم العودة إلى التلقائي
    else:  # الوضع AUTO
        auto_control()  # المروحة والإضاءة والري
    check_alarms()  # الإنذارات
    if loop % 5 == 0:  # الحالة كل 5 ثوانٍ
        show_status()  # Serial + Nextion
    loop = loop + 1  # الدورة التالية
    time.sleep(1)  # انتظار ثانية ثم التكرار
```

6. البرنامج كاملاً

esp/main.py

```
# COMPETITION MASTER 02 — ESP32 Complete Smart Farm (MicroPython)
# نفس منطق مشروع Raspberry Pi تماماً — الاختلاف فقط في طريقة الوصول إلى الأرجل
import time                                    # الوقت والانتظار (sleep)
import dht                                     # حساس الحرارة والرطوبة DHT22
from machine import Pin, ADC, UART             # الأرجل + محوّل ADC الداخلي + منافذ UART

# ---------- الإعدادات: نفس قيم مشروع Raspberry Pi (1 ----------
MAX_TEMP = 30                  # فوق 30 درجة: تشغيل المروحة
FAN_OFF_TEMP = 27              # تحت 27 درجة: إيقاف المروحة
MIN_WATER = 20                 # أقل مستوى ماء مسموح (%) لتشغيل المضخة
PUMP_SECONDS = 5               # مدة الري بالثواني
IRRIGATION_HOURS = [7, 12, 17] # ساعات الري في اليوم
LIGHT_ON_HOUR = 6              # ساعة تشغيل الإضاءة
LIGHT_OFF_HOUR = 20            # ساعة إطفاء الإضاءة
DOSE_A_ML = 10                 # جرعة السماد A بالمليلتر
DOSE_B_ML = 5                  # جرعة السماد B بالمليلتر
FLOW_ML_PER_SEC = 2            # تدفق مضخة الجرعات (يحتاج معايرة على الجهاز الحقيقي)
PH_MIN = 5.5                   # أقل pH مقبول
PH_MAX = 6.5                   # أعلى pH مقبول
EC_MIN = 1.0                   # أقل EC مقبول (mS/cm)
EC_MAX = 2.5                   # أعلى EC مقبول
CO2_MAX = 1500                 # أعلى CO2 مقبول (ppm)
MIN_PUMP_AMPS = 0.2            # أقل تيار يثبت أن المضخة تعمل فعلاً (أمبير)
TEST_SECONDS = 2               # مدة تشغيل كل جهاز في وضع الاختبار

# >>> PINS
# ---------- 2) التوصيلات: كل جهاز على رجله ----------
dht_sensor = dht.DHT22(Pin(4))                      # DHT22 :البيانات على GPIO4
UART2 = UART(2, baudrate=9600, tx=17, rx=16, timeout=300)  # MH-Z19B على co2_port (TX=17، RX=16)
screen = UART(1, baudrate=9600, tx=18, rx=19)       # شاشة Nextion على UART1 (TX=18، RX=19)
ph_adc = ADC(Pin(36), atten=ADC.ATTN_11DB)          # pH على GPIO36 (ADC1) يعمل مع Wi-Fi، حتى 3.3 فولت تقريباً
ec_adc = ADC(Pin(39), atten=ADC.ATTN_11DB)          # EC على GPIO39 (ADC1)
level_adc = ADC(Pin(34), atten=ADC.ATTN_11DB)       # مستوى الماء على GPIO34 (ADC1)
current_adc = ADC(Pin(35), atten=ADC.ATTN_11DB)     # تيار المضخة ACS712 على GPIO35 (ADC1)
pump = Pin(26, Pin.OUT, value=0)                    # مرحّل مضخة الري على GPIO26 (يبدأ مطفأ)
fan = Pin(27, Pin.OUT, value=0)                     # مرحّل المروحة على GPIO27
light = Pin(14, Pin.OUT, value=0)                   # مرحّل الإضاءة على GPIO14
alarm_lamp = Pin(13, Pin.OUT, value=0)              # مرحّل مصباح الإنذار على GPIO13
dose_a = Pin(25, Pin.OUT, value=0)                  # مرحّل مضخة الجرعات A على GPIO25
dose_b = Pin(33, Pin.OUT, value=0)                  # مرحّل مضخة الجرعات B على GPIO33
dosing_button = Pin(23, Pin.IN, Pin.PULL_UP)        # زر الجرعات على GPIO23 (إلى GND، مقاومة سحب داخلية)
test_button = Pin(32, Pin.IN, Pin.PULL_UP)          # زر الاختبار على GPIO32 (إلى GND)
# <<< PINS

# ---------- 3) حالة النظام ----------
mode = "AUTO"                  # الوضع الحالي: AUTO أو DOSING أو TEST
alarm = "NONE"                 # الإنذار الحالي
pump_failed = False            # يصبح True إذا شغّلنا المضخة ولم يمر تيار
temperature = None             # آخر قراءة حرارة
humidity = None                # آخر قراءة رطوبة
co2 = None                     # آخر قراءة CO2
ph = 0                         # آخر قراءة pH
ec = 0                         # آخر قراءة EC
water_level = 0                # مستوى الماء %
pump_amps = 0                  # تيار المضخة بالأمبير
last_irrigation_hour = -1      # آخر ساعة تم فيها الري (حتى لا نكرره)
start_time = time.time()       # وقت بدء التشغيل لحساب يوم النمو
last_dht_time = 0              # وقت آخر قراءة DHT22 (يحتاج ثانيتين بين القراءات)


def volts(adc):                # جهد مدخل ADC بالفولت
    return adc.read_uv() / 1000000  # تعطي ميكروفولت معايَراً


def read_co2():                # قراءة CO2 من MH-Z19B عبر UART
    co2_port.write(bytes([0xFF, 0x01, 0x86, 0, 0, 0, 0, 0, 0x79]))  # أمر طلب القراءة (9 بايت)
    reply = co2_port.read(9)   # ننتظر رداً من 9 بايت (أو None بعد المهلة)
    if reply is None or len(reply) != 9:  # لا رد = الحساس مفصول
        return None            # تعني: لا توجد قراءة
    return reply[2] * 256 + reply[3]  # البايت 2 و 3 = التركيز ppm

```

```
72
73  # >>> SENSORS
74  قراءة كل الحساسات #                                :()def read_sensors
75  سنغيّر هذه المتغيرات #  global temperature, humidity, co2, ph, ec, water_level, pump_amps, last_dht_time
    ↪ العامة
76      قراءة جديدة كل ثانيتين فقط :if time.time() - last_dht_time >= 2:            # DHT22
77          نحفظ وقت هذه القراءة #                 ()last_dht_time = time.time
78          نحاول القراءة #                                    :try
79              نطلب القياس أولاً MicroPython في #                    ()dht_sensor.measure
80              C° الحرارة #  ()temperature = dht_sensor.temperature
81              % الرطوبة #        ()humidity = dht_sensor.humidity
82          (الحساس مفصول) فشل القراءة #                              :except OSError
83              لا توجد قراءة حرارة #                      temperature = None
84              ولا رطوبة #                           humidity = None
85      (None أو) co2 = read_co2()                                  # CO2 ppm
86      الجهد × Raspberry Pi: pH = 3.5 نفس معادلة #             ph = round(volts(ph_adc) * 3.5, 2)
87      0.5 ÷ الجهد = ec = round(volts(ec_adc) / 0.5, 2)              # EC
88      %100 = فولت water_level = round(volts(level_adc) / 3.0 * 100)  # 3
89      صفر أمبير = فولت pump_amps = round((volts(current_adc) - 2.5) / 0.185, 2)  # ACS712: 2.5
90  # <<< SENSORS
91
92
93  هل يوجد ماء كافٍ؟ #                                      :()def water_ok
94      قراءة غير صالحة = %100 أكثر من #  return water_level >= MIN_WATER and water_level <= 100
95
96
97  يوم النمو منذ بدء التشغيل #                                    :()def growth_day
98      يوم واحد = ثانية return int((time.time() - start_time) / 86400) + 1  # 86400
99
100
101 # >>> IRRIGATION
102 مع الحماية PUMP_SECONDS ري لمدة #                                  :()def irrigate
103     قد نغيّر هذا المتغير العام #                               global pump_failed
104     رسالة بداية الري #                        print("IRRIGATION start")
105     ثانية بعد ثانية #              :for second in range(PUMP_SECONDS)
106         قراءة جديدة قبل كل ثانية #                              ()read_sensors
107         الماء نقص أثناء الري #                          :()if not water_ok
108             نخبر بالسبب #  print("IRRIGATION stopped: LOW WATER")
109             نخرج من الحلقة فوراً #                                  break
110         تشغيل المضخة #                                  ()pump.on
111         انتظار ثانية #                                time.sleep(1)
112         نقرأ تيار المضخة الآن #                                 ()read_sensors
113         المضخة لا تسحب تياراً = لا تعمل #               :if pump_amps < MIN_PUMP_AMPS
114             نتذكر العطل (يظهر الإنذار) #                    pump_failed = True
115             رسالة العطل #       print("PUMP FAILURE: no current")
116             نتوقف عن الري #                                  break
117     إيقاف المضخة دائماً في النهاية #                                       ()pump.off
118     رسالة نهاية الري #                          print("IRRIGATION end")
119 # <<< IRRIGATION
120
121
122 # >>> DOSING
123 (لا تعملان معاً أبداً) B ثم جرعة A جرعة #                                    :()def run_dosing
124     لا تسميد بدون ماء #                                :()if not water_ok
125         نخبر بالسبب #        print("DOSING cancelled: LOW WATER")
126         نخرج من الدالة #                                    return
127     A متوقفة قبل تشغيل B تأكد أن #                                  ()dose_b.off
128     A تشغيل #                                     ()dose_a.on
129     A رسالة الجرعة #              print("DOSING A", DOSE_A_ML, "ml")
130     (ثوانٍ 5 = 2 ÷ 10) الزمن = الكمية ÷ التدفق #         time.sleep(DOSE_A_ML / FLOW_ML_PER_SEC)
131     A إيقاف #                                     ()dose_a.off
132     B ثم تشغيل #                                     ()dose_b.on
133     B رسالة الجرعة #              print("DOSING B", DOSE_B_ML, "ml")
134     ثانية time.sleep(DOSE_B_ML / FLOW_ML_PER_SEC)         # 5 ÷ 2 = 2.5
135     B إيقاف #                                     ()dose_b.off
136     انتهت الجرعات #                               print("DOSING DONE")
137 # <<< DOSING
138
139
140 تجربة كل جهاز لثانيتين #                                      :()def run_test
141     قائمة #  :for name, device in [("FAN", fan), ("LIGHT", light), ("DOSE A", dose_a), ("DOSE B", dose_b)]
    ↪ الأجهزة
142         اسم الجهاز الذي نختبره #                          print("TEST", name)
143         تشغيله #                                  ()device.on
144         ننتظر ثانيتين #                     time.sleep(TEST_SECONDS)
145         إيقافه #                                  ()device.off
146 (تُلغى لا الحماية) ماء يوجد إذا فقط المضخة #                                  :()if water_ok
```

```
147 نختبر المضخة #                          print("TEST PUMP")
148 تشغيلها #                                  ()pump.on
149 ننتظر ثانيتين #                    time.sleep(TEST_SECONDS)
150 إيقافها #                                 ()pump.off
151 لا يوجد ماء كافٍ #                                    :else
152 لا نشغّل المضخة فارغة #       print("TEST PUMP skipped: LOW WATER")
153 انتهى الاختبار #                           print("TEST DONE")
154
155
156 الوضع التلقائي #                              :()def auto_control
157 سنغيّر هذا المتغير العام #                  global last_irrigation_hour
158 3 هي العنصر رقم MicroPython في الساعة الحالية: #                    hour = time.localtime()[3]
159     # >>> FAN
160 لا قراءة حرارة: نشغّل المروحة للأمان #                       :if temperature is None
161 تشغيل المروحة #                                ()fan.on
162 حار جداً #                    :elif temperature > MAX_TEMP
163 تشغيل المروحة #                                ()fan.on
164 برد بما يكفي #                :elif temperature < FAN_OFF_TEMP
165 (بين 27 و 30 تبقى كما هي) إيقاف المروحة #                                ()fan.off
166     # <<< FAN
167     # >>> LIGHT
168 داخل ساعات الإضاءة #  :if hour >= LIGHT_ON_HOUR and hour < LIGHT_OFF_HOUR
169 تشغيل الإضاءة #                                 ()light.on
170 خارج ساعات الإضاءة #                                    :else
171 إطفاء الإضاءة #                                ()light.off
172     # <<< LIGHT
173 حان وقت ري جديد #  :if hour in IRRIGATION_HOURS and hour != last_irrigation_hour
174 نتذكر الساعة حتى لا نكرر الري #                last_irrigation_hour = hour
175 يوجد ماء؟ #                             :()if water_ok
176 نروي #                            ()irrigate
177 لا يوجد ماء #                                  :else
178 نتخطى الري #  print("IRRIGATION skipped: LOW WATER")
179
180
181 # >>> ALARMS
182 اختيار أهم إنذار وتشغيل المصباح #                              :()def check_alarms
183 سنغيّر هذا المتغير العام #                                  global alarm
184 نحفظ الإنذار السابق #                                   old = alarm
185 قراءة غير صالحة #  :if temperature is None or co2 is None or ph < 1 or ph > 14 or water_level > 100
186 خطأ حساس #                      "alarm = "SENSOR ERROR
187 الماء قليل #                             :()elif not water_ok
188 نقص الماء #                         "alarm = "LOW WATER
189 المضخة لم تسحب تياراً #                                :elif pump_failed
190 عطل المضخة #                       "alarm = "PUMP FAILURE
191 حرارة عالية #                     :elif temperature > MAX_TEMP
192 ارتفاع الحرارة #                  "alarm = "HIGH TEMPERATURE
193 خارج الحدود #  :elif ph < PH_MIN or ph > PH_MAX or ec < EC_MIN or ec > EC_MAX or co2 > CO2_MAX
194 قيمة خارج المجال #                       "alarm = "OUT OF RANGE
195 كل شيء طبيعي #                                       :else
196 لا يوجد إنذار #                             "alarm = "NONE
197 لا إنذار #                               :"if alarm == "NONE
198 إطفاء مصباح الإنذار #                                ()alarm_lamp.off
199 يوجد إنذار #                                       :else
200 تشغيل مصباح الإنذار #                                ()alarm_lamp.on
201 نطبع الإنذار عند تغيّره فقط #                                 :if alarm != old
202 رسالة الإنذار #                        print("ALARM", alarm)
203 # <<< ALARMS
204
205
206 لعرض حالة جهاز "OFF" أو "def on_off(device):                                  # "ON
207 0 تعطي 1 (يعمل) أو ()if device.value() == 1:                           # Pin.value
208 يعمل #                                   "return "ON
209 متوقف #                                      "return "OFF
210
211
212 Nextion كتابة نص في عنصر على شاشة #                                :def show(name, text)
213 "tTemp.txt="24.5 :مثال #  screen.write(name + '.txt="' + str(text) + '"')
214 0xFF ينتهي بثلاثة Nextion كل أمر #         screen.write(bytes([0xFF, 0xFF, 0xFF]))
215
216
217 وعلى الشاشة Serial الحالة على #                                  :()def show_status
218     print("STATUS mode=" + mode, "temp=" + str(temperature), "hum=" + str(humidity), "co2=" + str(co2),  #
 ↪ الوضع والهواء
219 الماء #  ,ph=" + str(ph), "ec=" + str(ec), "level=" + str(water_level), "pump=" + on_off(pump)"
 ↪ والمضخة
220           "fan=" + on_off(fan), "light=" + on_off(light), "alarm=" + alarm, "day=" + str(growth_day()))  #
 ↪ الباقي
```

```
                    show("tTemp", temperature)  # الحرارة على الشاشة
                  show("tHum", humidity)  # الرطوبة
    show("tCO2", co2)                           # CO2
    show("tPH", ph)                             # pH
    show("tEC", ec)                             # EC
                 show("tLevel", water_level)  # مستوى الماء
                   show("tMode", mode)  # الوضع
                   show("tAlarm", alarm)  # الإنذار
                show("tDay", growth_day())  # يوم النمو


---------- الحلقة الرئيسية (4 ---------- #
print("Smart farm started (ESP32)")  # رسالة البداية على Serial Monitor
       loop = 0  # عدّاد الدورات
      :while True  # تكرار إلى الأبد
                                    # 1  قراءة الحساسات (read_sensors()
                                    # 2  الزر المضغوط يقرأ 0 (موصول إلى GND) (if dosing_button.value() == 0:
            "mode = "DOSING  # نختار وضع الجرعات
        :if test_button.value() == 0  # زر الاختبار مضغوط؟
              "mode = "TEST  # نختار وضع الاختبار
                                 # 3  تنفيذ الوضع (if mode == "DOSING":
            ()run_dosing  # الجرعات A ثم B
              "mode = "AUTO  # ثم العودة إلى التلقائي
    :"elif mode == "TEST  # وضع الاختبار
            ()run_test  # تجربة الأجهزة
              "mode = "AUTO  # ثم العودة إلى التلقائي
              :else  # الوضع AUTO
           ()auto_control  # المروحة والإضاءة والري
                # 4  الإنذارات (check_alarms()
                                    # 5  الحالة كل 5 ثوانٍ (if loop % 5 == 0:
    show_status()                  # Serial + Nextion
 loop = loop + 1  # الدورة التالية
                                    # 6  انتظار ثانية ثم التكرار (time.sleep(1)

```

7. سيناريوهات الاختبار ونتائجها

نتائج السيناريوهات التي شُغّلت في المتصفح بتاريخ 05-11-2026 (بيئة التشغيل الحقيقية في المحاكي).

السيناريو	ماذا يفعل	المتوقع	النتيجة
1 · التشغيل العادي	الساعة 08:00، الحرارة 25 °م، الماء 75٪: شغّل 12 ثانية دون أي تدخل.	أسطر STATUS بكل القراءات، الوضع AUTO، الإضاءة تعمل (06–20)، المروحة والمضخة متوقفتان، الإنذار NONE، مصباح الإنذار مطفأ.	PASS
2 · ارتفاع الحرارة (25 ← 35 °م)	عند 3 ث تصبح الحرارة 35 °م، وعند 9 ث تصبح 28.5 °م (بين الحدين)، وعند 15 ث تصبح 25 °م.	تعمل المروحة ويظهر إنذار HIGH TEMPERATURE فوق 30 °م؛ عند 28.5 °م يزول الإنذار وتبقى المروحة تعمل (هامش التخلّف)؛ وتحت 27 °م تتوقف المروحة.	PASS
3 · انخفاض الماء	الساعة 11:59:55 — عند 2 ث ينخفض الخزان إلى 10٪؛ وموعد ري 12:00 بعد 3 ثوانٍ.	إنذار LOW WATER، مصباح الإنذار يعمل، و"IRRIGATION skipped: LOW WATER": المضخة لا تعمل أبداً.	PASS
4 · الري الساعة 12:00	ابدأ الساعة 11:59:56 وشغّل 25 ثانية.	"IRRIGATION start"، تعمل المضخة نحو 5 ثوانٍ (مع قياس التيار كل ثانية)، ثم "IRRIGATION end"، ومرة واحدة فقط في تلك الساعة.	PASS
5 · الجرعات A ثم B	عند 2 ث اضغط زر DOSING لمدة 1.5 ثانية.	DOSING A 10 مل (5 ث)، ثم DOSING B 5 مل (2.5 ث)، ولا تعملان معاً أبداً، ثم DOSING DONE والعودة إلى AUTO. الكميات من نموذج المضخات في المزرعة (2 مل/ث).	PASS
6 · وضع الاختبار	الساعة 21:00 (الإضاءة مطفأة حسب الجدول): عند 2 ث اضغط زر TEST لمدة 1.5 ثانية.	TEST FAN ثم TEST LIGHT ثم TEST DOSE A ثم DOSE B ثم TEST PUMP (لأن الماء كافٍ فقط)، ثم TEST DONE؛ يعمل كل حمل مرة واحدة، ثم يطفئ AUTO الإضاءة من جديد.	PASS
7 · عطل حساس	عند 3 ث يُضبط عطل "Sensor disconnected" على حساس الهواء (DHT22)؛ وعند 10 ث يُزال العطل.	إنذار SENSOR ERROR، وقيمة temp=None/null في STATUS، وتعمل المروحة (الجانب الآمن) ما دامت القراءة مفقودة؛ ويزول الإنذار بعد الإصلاح.	PASS
8 · جدول الإضاءة	ابدأ الساعة 19:59:50 وشغّل 18 ثانية.	الإضاءة تعمل قبل 20:00 وتنطفئ بعد 20:00 (LIGHT_OFF_HOUR).	PASS
9 · خطأ توصيل (سلك المضخة مفصول)	يُزال السلك من تلامس المرحل NO1 إلى ACS712، ثم يبدأ ري 12:00 (البداية 11:59:56).	يعمل المرحل لكن لا يمر تيار: "PUMP FAILURE: no current"، وإنذار PUMP FAILURE، ويُطفأ المرحل فوراً.	PASS
10 · النظام الكامل	الساعة 11:59:52: ري الساعة 12:00، وعند 14 ث تصل الحرارة إلى 35 °م، وعند 18 ث اضغط زر DOSING لمدة 1.5 ثانية، وعند 32 ث تعود الحرارة إلى 25 °م.	ري (نحو 5 ث)، ثم المروحة وإنذار HIGH TEMPERATURE، ثم الجرعات A ثم B، ثم تزول كل الإنذارات وتتوقف المروحة: يعود النظام إلى AUTO العادي.	PASS

8. غيّر شيئاً واحداً — تمارين

1. غيّر حد تشغيل المروحة

   اجعل MAX_TEMP = 28 وFAN_OFF_TEMP = 25. شغّل، ثم اضبط حرارة هواء المزرعة على 29 °م.

   ما يجب أن تلاحظه: تعمل المروحة الآن عند 29 °م ويظهر إنذار HIGH TEMPERATURE.

2. غيّر مدة الري

   اجعل PUMP_SECONDS = 8 وأضف الساعة الحالية إلى IRRIGATION_HOURS. شغّل وراقب المضخة.

   ما يجب أن تلاحظه: يظهر IRRIGATION start ثم IRRIGATION end بعد نحو 8 ثوانٍ؛ ويُظهر تدفق المزرعة عمل المضخة.

3. انقل المروحة إلى GPIO آخر

   انقل السلك من GPIO27 إلى GPIO21 وغيّر Pin(27, ...) إلى Pin(21, ...). لا تستخدم GPIO34–39 (للإدخال فقط).

   ما يجب أن تلاحظه: تعمل المروحة من الرجل الجديدة. إذا تغيّر الكود فقط أو السلك فقط تبقى المروحة متوقفة بينما يظن الكود أنها تعمل (GPIO27).

4. افصل حساس المستوى

   أثناء عمل البرنامج احذف سلك إشارة حساس مستوى الماء.

   ما يجب أن تلاحظه: يُقرأ المستوى بشكل خاطئ (قريباً من 0): إنذار LOW WATER، يضيء المصباح، ويُرفض الري والجرعات. أعد السلك (Ctrl+Z).

5. غيّر حجم الجرعة

   اجعل DOSE_A_ML = 20 وDOSE_B_ML = 10، ثم ابدأ DOSING.

   ما يجب أن تلاحظه: تعمل A 10 ثوانٍ وB 5 ثوانٍ؛ وتُظهر المزرعة نحو 20 مل و10 مل. تغيير FLOW_ML_PER_SEC دون تغيير المضخة يغيّر الزمن فقط وليس التدفق الحقيقي — لذلك المعايرة مهمة.

6. عطّل المضخة أثناء التشغيل

   اختر المضخة الرئيسية، واضبط عطلها على "Pump failure"، وابدأ الري (ضع الساعة الحالية في IRRIGATION_HOURS).

ما يجب أن تلاحظه: يغلق المرحّل لكن ACS712 لا يقيس تياراً: PUMP FAILURE: no current، وإنذار PUMP FAILURE.

7. غيّر وقت الإضاءة

اجعل LIGHT_ON_HOUR = 8 وLIGHT_OFF_HOUR = 18. استخدم الساعة (التسريع) لتجاوز 18:00.

ما يجب أن تلاحظه: تنطفئ الإضاءة الساعة 18:00 بدلاً من 20:00.

8. ضيّق مجال pH

اجعل PH_MAX = 6.0 (محلول المزرعة نحو pH 6.2).

ما يجب أن تلاحظه: يظهر ALARM OUT OF RANGE ويضيء المصباح؛ ولا يتغير شيء آخر لأن OUT OF RANGE تحذير فقط.

9. التنفيذ على العتاد الحقيقي

1. ثبّت MicroPython على ESP32 (esptool.py --chip esp32 write_flash -z 0x1000 ESP32_GENERIC-....bin).
2. انسخ البرنامج باسم main.py باستخدام Thonny أو mpremote (mpremote cp esp/main.py :main.py).
3. غذِّ المرحّلات والحساسات وNextion من مصدر 5 فولت منفصل واربط أرضيه بأرضي ESP32.
4. مداخل ADC في ESP32 تقبل نحو 3.1 فولت على الأكثر مع ATTN_11DB: خرج ACS712 (2.5 فولت + 0.185 فولت/أمبير) يبقى تحتها حتى نحو 3 أمبير؛ استخدم مقسّم جهد للمضخات الأكبر.
5. حمّل صفحة Nextion (مربعات النص tDay ... tTemp) إلى الشاشة باستخدام Nextion Editor (ملف .tft).
6. اضبط ساعة ESP32 (ntptime) عبر Wi-Fi أو (RTC): الجدول يستخدم time.localtime().

10. المكتبات المطلوب تثبيتها

- MicroPython firmware (machine, dht, time built in): https://micropython.org/download/ESP32_GENERIC/
- mpremote / Thonny: pip install mpremote
- Nextion Editor (screen design): Windows application from nextion.tech

11. الفروقات بين المحاكاة والجهاز الحقيقي

- يعمل MicroPython 1.29 فعلاً (WebAssembly)؛ أما ملحقات الشريحة فمحاكاة: قيم ADC تأتي من حل الدائرة وليس من محوّل حقيقي بعدم خطيته.
- Nextion نموذج تدريبي لمجموعة الأوامر؛ لا تُستخدم ملفات HMI/.TFT..
- ساعة المحاكي مضبوطة لك؛ ESP32 الحقيقي يبدأ من 2000-01-01 حتى تضبط الوقت.
- لا يستخدم هذا البرنامج Wi-Fi ولا خادم ويب (ولا ندّعي ذلك).

هذه المشاريع مراجع تدريبية؛ ولا تثبت المعدات التي ستُستخدم في النهائي ولا تغطي كل الوحدات التنافسية.